Mastercard Innovation Challenge 2026

Team - Chakravyuh

Closed-Loop Adversarial Curriculum Learning for Payment Fraud Defense

Members - Sneh Kansagara, Priyanshu Jha

Summary

Most fraud-simulation submissions create synthetic fraud, train a model on it, and then report a 0.98 AUC. That is circular because the same team designed both sides of the setup. Chakravyuh is designed to avoid that trap.
Attacks come from real payment-rail trust anchors instead of being imagined. The legitimate background population is built and validated before any attacks are introduced. One attack family is kept completely out of training. Precision is measured at a fixed, tight false-positive rate. The closed loop, where detector misses drive new attack generation, is evaluated end-to-end. This includes a real regression that was caught and fixed during development rather than just showing the loop in a diagram.

Headline evidence

Attacks	Features	Test PR-AUC	Held-out recall	Cross-gen evasion	p99 latency	Lookalike gap
16 families (58 catalogue entries)	81	0.9987	100% (609/609)	<1%, all 3 generations	<2ms, CPU-only	2.12× (target 1.0×, open)

1. Problem & Motivation

GenAI does not create new payment rails to attack. Instead, it makes the human-facing steps of existing attacks almost free, such as cloned voices from 3–10s of audio, personalised scam scripts at scale, and synthetic KYC/KYB documents generated in minutes. Rating every UPI trust anchor on a four-level ladder (V3 cryptographic, V2 probabilistic/ML-scored, V1 self-asserted, V0 unverified) surfaced the finding that shapes this whole system. The UPI PIN proves that someone who knew it pressed the keys on the bound device, but it proves nothing about whether they understood what they were authorising. Every major Indian scam pattern, including digital arrest, KYC expiry, romance, and job-task scams, converges on exactly this gap. A V3 mechanism is being used to support a V0 inference. This is why the system is framed as intent detection rather than anomaly detection, and why the schema in Section 4 includes session and coercion fields such as time_on_confirm_screen_s, screen_share_active, and call_active_during_txn that a normal transaction table would leave out.

- UPI credits are irreversible, and there is no chargeback. Detection therefore has to happen before authorisation and within a sub-second window, making it a control rather than a reporting tool.
- Authorised-but-deceived payments fall outside India's zero-liability framework, which covers only unauthorised transactions. That creates a real structural gap and is part of why there is already little economic pressure to solve it.

A linear model can weight screen_share_active, call_active_during_txn, and beneficiary_first_time independently, but it cannot combine them into the interaction that signals coercion. Section 6 confirms this empirically through the wide PR-AUC gap between Logistic Regression and tree models on the same features.

2. System Overview

Three pillars share one canonical data contract, plus a feedback path that closes the loop between them: src/ (Identify + Generate), stage5/ (Defend + closed loop), web/ (prototype).

#	Stage	What happens
1	Identify	Trust-map derivation across 5 rail phases → 58 catalogue entries → 16 code-implemented generator families.
2	Generate (base)	Legitimate parties/devices/merchants and calibrated background traffic, validated before any attack exists.
3	Generate (attack)	16 generators inject labelled fraud campaigns + matched legitimate lookalikes into the same schema.
4	Feature engineering	81 features (categorical/numerical/boolean/behavioural/graph), temporal 60/20/20 split.
5	Defend	XGBoost (150 trees, depth 6), evaluated at fixed 0.1%/1% FPR against LR/RF baselines.
6	Explain & mitigate	SHAP attribution per transaction, plus an optional GenAI analyst narrative with a template fallback.
7	Evaluate	Cross-generation regression testing, held-out-family generalisation, lookalike-fidelity check.
8	Feedback	Deployed model's feature_importances_ auto-reconfigure the attack generator, Gen 3→4→5 curriculum retrains.

3. Identify: Attack Intelligence Layer

The process was to first break each rail into lifecycle phases (initiate→authenticate→authorise→settle→dispute), then map who trusts whom and the assumption behind that trust. Each trust anchor was then challenged by asking whether the human could be manipulated, whether the mechanism could be fooled or flooded, whether the signal was probabilistic, and what GenAI had made cheaper. Every flow was then replayed for each actor, including the outsider, deceived customer, fraudulent or compromised merchant, insider, AI agent, and coordinated network.
Finally, the flows were inverted to find attacks that keep every conventional signal looking normal. This is where novelty lives.

The raw pass produced 58 catalogue entries across card, UPI, IMPS-NEFT-RTGS, wallets, and BNPL. Entries with the same data signature were collapsed into a single generator with a pretext parameter. For example, digital-arrest, KYC-expiry, and romance scams all show a genuine device, correct PIN, new beneficiary, and coercion fields. All rows are kept for the diversity count, but only one implementation is used per signature. Three more families were added from documented mechanisms rather than catalogue entries. Final count: 16 code-implemented generators, each emitting its own labelled campaign along with a matched legit_lookalike population.

16 attack-generator families (src/attacks/generators.py)

Generator	Rail	Mechanism	Detection signal
scam_ induced_ push	UPI P2P	Victim manipulated into authorising a genuine, correctly-PIN'd transfer.	Session/behavioural anomaly + rapid beneficiary add
mule_ network	UPI/IMPS	Funds layered through an account chain to obscure destination.	Graph fan-in/fan-out, pass-through ratio
card_ testing_ probe	Card CNP	LLM-optimised brute-force card validity probing.	Velocity (16+/10min) + MCC diversity + declines
adversarial_ evasion	UPI/Card	Fraud built to sit inside the legitimate distribution, evading the current model.	Near-legitimate ranges by construction
first_ party_ dispute	Card CNP	Holder disputes own genuine transaction as chargeback.	Dispute filed within 60 seconds of the transaction, based only on dispute history
stealth_ mandate	UPI mandate	Unauthorised recurring mandate drawn down below alert thresholds.	max_amount ≫ actual_amount
synthetic_ merchant	Card/P2M	Fake merchant funnels funds before disappearing.	Merchant velocity + fast settlement outflow
transaction_ laundering	Card/P2M	Transactions structured/miscoded to evade thresholds.	Declared vs. inferred MCC mismatch
credential_ takeover	UPI/Card	Compromised account used normally, then drained.	Device/geo mismatch + velocity spike
synthetic_ identity_ bustout	Card CNP	Credit-building then sudden utilisation spike. Held out of training (Section 6).	30-day dormancy then 5–10 high-value txns
subthreshold_ fragmentation	UPI Lite/Wallet	Sub-threshold transactions to the same payee.	10+ txns to same destination in 2h

agentic_ injection	UPI/agentic	LLM agent manipulated into unauthorised transfer.	is_agent_initiated, and the beneficiary is not the seller of record
insider_ abuse	Card CP/Bank	Employee processes fraud via legitimate access.	Unusual time-of-day + override flags
device_ fan_ out	Card CP	One device drains 4–6 cards in a 2h window.	Device-centric clustering
balance_ drain_ exit	UPI P2P	Large inbound transfer, 85–95% payout within 5min.	Rapid receiver balance collapse
tpap_ account_ switch	UPI (cross-app)	One UPI handle drained across 3–5 linked accounts, rotating apps.	Distinct-TPAP velocity, invisible to any single bank

Diversity, precisely scoped: 58 catalogue entries (distinct narratives) → 16 generator families (distinct data signatures, which is what the detector actually learns) → 4 curriculum difficulty levels each. Coverage spans card CNP/CP, all UPI sub-rails, IMPS/NEFT/RTGS, wallets, BNPL, cross-app and agentic flows across every lifecycle phase. Three gaps are reported as structural rather than missing work. UPI has no consumer dispute remedy, card-present has no exploitable initiate-phase exchange, and agentic settlement/dispute infrastructure does not exist yet. Every entry is tagged Observed / Emerging / Speculative based on a dated-source evidence pass. Two entries were downgraded, and one unverifiable statistic was removed instead of being kept just because it sounded precise. See docs/attack-catalogue.md and docs/research/evidence-pass.md.

The inversion pass - the novelty argument for Identify

Conventional models assume the attacker is not the customer. In a scam-induced payment, everything is genuine except the intent, and no standard column captures that. Backed by the measured gap between Logistic Regression and tree models (Section 5).

4. Generate: Attack Simulation Engine

The legitimate base population, including parties, devices, merchants, and background traffic, is built and statistically validated first. If the legitimate population is crude, an attack does not need to be realistic to stand out. The detector ends up separating two trivially different distributions instead of actually catching fraud. The generator and detector use the same seven-table schema (transactions, parties, merchants, mandates, disputes, graph_edges, labels) under one rule: only include a field that a real payment system would have at decision time. The highest-value fields, time_on_confirm_screen_s, screen_share_active, call_active_during_txn, and beneficiary_added_ago_s, are the only place where the intent signal becomes measurable. has_salary_credit, organic_spend_ratio, and graph fan-in/out are the strongest mule discriminators. Every generator also emits a matched legit_lookalike population with the same rail, amount, and pattern, but genuinely not fraud. This is enforced through test coverage rather than being optional, because without it, a classifier can separate two trivially different distributions and report a meaningless 0.99 AUC.

5. Defend & Experimental Setup

81 features (13 categorical, 10 numerical, 10 boolean, 43 behavioural, 5 graph) are computed only from the raw tables. They never use information that a live system would not have. The data is then split 60/20/20 by transaction timestamp, never randomly. train_fraud_model.py fits Logistic Regression, Random Forest, and XGBoost side by side on the identical split and selects the winner based on validation PR-AUC. The deployed model is XGBoost with 150 trees, depth 6, a learning rate of 0.1, and scale_pos_weight set from the actual class ratio, with fraud prevalence well under 1%.

- **Latency:** pre-auth, sub-second UPI decisioning. 150 shallow trees score in microseconds.
- **Class imbalance:** scale_pos_weight directly reweights the loss, while Random Forest's class_weight uses a coarser per-tree adjustment.
- **Explainability:** fast, mature SHAP (TreeExplainer) support for the analyst layer in Section 10.
- **Calibrated operating points:** probability outputs hold up better under fixed-FPR threshold sweeps than Random Forest's vote-fraction estimates.

6. Results

Numbers below are read directly from the pipeline's own on-disk artifacts (stage5/models/model_metadata.json, stage5/validation/cross_generation_evaluation.json), current as of the latest training run.

Operating point	Achieved FPR	Precision	Recall

Target 0.1% FPR	0.099%	0.9638	0.9917
Target 1% FPR	0.594%	0.8169	0.9994
F1-optimal threshold	0.206%	0.9275	0.9942 (F1 0.9597)

Test PR-AUC 0.9987, ROC-AUC 0.99995 (secondary), and Brier 0.0123. Held-out synthetic_identity_bustout, which was never seen during training or validation, had 609/609 cases caught (100%, 95% CI [0.994, 1.0]) at both FPR points. Read together with the lookalike-fidelity FAIL in Section 4, this should be treated as an upper bound on generalisation, not a calibrated estimate.

Cross-generational robustness - deployed model vs. held-out attacks from all 3 curriculum generations

Generation	Evasion	Caught / total	Target	Status
Gen 3 - feature-hiding	1.1%	2,721 / 2,750	<5%	PASS
Gen 4 - ensemble-trading	0.1%	2,298 / 2,300	<5%	PASS
Gen 5 - multi-family cross	0.2%	1,597 / 1,600	<5%	PASS

Baseline comparison (validation split)

Model	Validation PR-AUC
Logistic Regression	0.8946
Random Forest	0.9769
XGBoost (deployed)	0.9968

7. Ablations & Error Analysis

Stress-testing the deployed model - surfaced real, specific weaknesses, reported in full rather than summarised away.

- **What the model leans on.** Feature importance is spread across plausible signals rather than one dominant proxy (beneficiary_added_ago_s 11.5%, session_duration_s 10.9%, txn_count_last_24h 9.5%). This is direct evidence that the Section 4 leak fixes worked.
- **Mule-network blind spot.** A synthetic transaction with both textbook mule signatures (is_two_hop_passthrough=1.0, payee_in_degree=85) but an unremarkable payer out-degree (28) scored 0.02% fraud probability, making it essentially undetected. A rule-based threshold was investigated and rejected. Legitimate merchant fan-in has a median of 266 and a maximum above 10,800, while two-hop occurs in 16.9% of all legitimate edges. The honest fix, class-weighted sampling or feature-masking on mule_network rows, is deferred rather than silently shipped as solved.
- **No safe single-transaction fix for 3 families.** first_party_dispute only exists after settlement. stealth_mandate depends on amount escalation across weeks rather than a single transaction. device_fan_out and balance_drain_exit require cross-transaction context. None has a safe single-transaction fix. The schema's own detectable_at field predicts this.
- **Velocity-feature starvation in the live demo.** txn_count_last_1h and txn_count_last_24h, the two highest-importance features, fall back to training-set medians in the live single-shot /api/analyze demo because there is no real-time feature store. As a result, two genuinely different inputs can score within about 0.5 percentage points of each other, even though the model is highly discriminative with full context at training time, as shown by the PR-AUC of 0.9987.
- **Rule overrides.** Eight families received targeted single-transaction policy overrides in the inference pipeline. Each was checked against legitimate calibration constants. This is a documented mitigation layer, not a substitute for the real-time feature store above.

8. Closed-Loop Red/Blue Team System

“Attacks become training data for the defence, and the defence’s blind spots generate new attacks” is usually a diagram. Chakravyuh implements it as two measured, executable mechanisms.

- **Mechanism 1 - adaptive attack config.** build_adaptive_attack_config.py loads the currently saved model,

checks feature_importances_ against a 10% threshold, and derives config for adversarial_evasion from whatever clears it (adaptive_top_counterparty, beneficiary_age_floor_s) - degrading gracefully to static defaults when no model exists yet. No manual step.

Generation	Config	Test PR-AUC	Recall @ 0.10% FPR
Gen 1 (no prior model)	Static defaults	0.9972	0.9942
Gen 2 (adaptive)	adaptive_top_counterparty=True, beneficiary_age_floor_s=50,578,560 (about 585 days)	0.9997	1.0000

- **Mechanism 2 - 3-generation adversarial curriculum.** Gen 3 (feature-hiding, 4 difficulty levels) → Gen 4 (ensemble-trading, hiding some features while exposing others) → Gen 5 (multi-family cross-attacks, 3+ families combined), each keeping prior-generation samples so the model does not forget what it already learned. Cross-generation evaluation gates every promotion.

9. Real-World Deployment & Feasibility

Design target, driven by the irreversibility finding in Section 1: pre-authorisation, sub-second control rather than post-hoc reporting. The 450KB XGBoost model scores in well under 1ms on CPU-only hardware, with p99 below 2ms, fitting comfortably within an issuer's authorisation window.

- **Scale & cost.** At 1M txns/day and 0.1% FPR, about 1,000 legitimate txns/day would queue for review. This provides a concrete operating point rather than hiding the trade-off behind an averaged accuracy number. A full Gen 3→4→5 retrain completes in about 2h on 8-core/14GB hardware. Inference is stateless and horizontally scalable, with no session store on the scoring path.
- **Red-teamed our own live API.** Probing /api/analyze found a real weakness. With no rate limiting, full-precision probabilities, and complete SHAP returned on every call, the endpoint gave enough information to work out the decision boundary just by sending repeated queries. A live CORS misconfiguration (allow_origins="*" with allow_credentials=True) found in the same pass was fixed immediately. Rate limiting has since been added to /api/analyze, with tests confirming it works. Full-precision probabilities and complete SHAP are still shown on every call, because full SHAP is a core part of the public demo. A more advanced setup, where public callers see less detail than the demo UI, is the next production-hardening step.

10. Working Prototype

A live analyst portal, not mockups, is available at chakravyuh-web.chatbot-sockscarving.workers.dev. It includes Risk Scoring Studio for live transaction scoring, SHAP, and an optional GenAI narrative, Closed-Loop Intelligence with the actual metrics from Sections 7 and 9 rather than illustrative placeholders, Attack Connection Graph with the catalogue mapped to lifecycle phases, and a dashboard and analyst-feedback surface for the human-in-the-loop path. The user journey is straightforward: discover an attack family → score a transaction → inspect SHAP and narrative → review closed-loop metrics → review analyst feedback surfaces. Every number shown comes from the same on-disk artifacts cited in Sections 7–9. The GenAI narrative is optional, disabled by default, and never part of the scoring decision.

11. Novelty & Differentiation

XGBoost is not claimed to be novel (Section 5). The novelty claims here are limited to what the evidence in this document actually supports:

- **Intent detection, not anomaly detection.** Conventional models assume the attacker is not the customer. In a scam-induced payment, everything is genuine except the intent, and intent does not appear in any standard column. This is backed by the measured gap between Logistic Regression and tree models (Section 5), not just claimed.
- **Closed-loop architecture, measured.** Two working feedback mechanisms (Section 9), backed by real before-and-after numbers from an actual retrain.
- **A safety gate that can catch its own failures.** cross_generation_eval.py caught a real 34.4%→43.1% regression, found the cause, and confirmed the fix. This validation gate fired for real during the 18-day deadline, not just in theory.
- **Systematic attack-derivation methodology.** The 16 families come from a systematic trust-anchor process (the

verification ladder, the inversion pass), not brainstormed ideas. The catalogue also checks its own research in Section 3, instead of treating the first draft as correct.
- **Self-directed red-teaming of the submission's own infrastructure.** The team red-teamed its own live inference API (Section 10), found a genuine model-extraction weakness, and fixed a live CORS bug on the spot.

12. Future Work

- **Lookalike fidelity.** The current evaluation shows a 2.12× separation ratio against the 1.0× target. The cause is known: synthetic pre-history is not yet threaded through campaign generation. The real fix needs a re-architecture, deliberately left for after submission.
- **Mule-network blind spot.** A campaign that keeps the payer's out-degree looking normal still gets past detection, scoring just 0.02% fraud probability. The fix is class-weighted retraining and more explicit network-level features.
- **Structurally undetectable families.** first_party_dispute, stealth_mandate, device_fan_out, and balance_drain_exit all depend on cross-transaction or post-settlement information a stateless model cannot see. This is a genuine boundary of the current detector, not a tuning problem.
- **AutoPay dark-pattern renewal.** A GenAI-era attack using a mandate-renewal message that hides an extension, identified in research but not implemented. Kept as a documented finding, not a weakly-grounded build.

13. Conclusion

Chakravyuh treats the brief's central risk, one team building both attacker and defender, as something to test, not assume away. Attacks are derived systematically and checked against real sources in Section 3. When the generator's own numbers looked too good, three real leaks were found and fixed, not accepted at face value, in Section 4. The detector reaches 0.9987 test PR-AUC and 100% recall on the held-out family in Section 7, but the lookalike-fidelity check missed its own target in Section 4, so that result is an upper bound, not a calibrated estimate.

The closed loop is measured, not just diagrammed: a live feature-importance feedback path, a 3-generation curriculum, and a safety check that caught and fixed a real regression (evasion rose from 34.4% to 43.1%) in Section 9. The prototype runs with sub-millisecond CPU-only inference. What production still needs (a real-time feature store, mule-network retraining, further API hardening) was found through the system's own instrumentation in Sections 8 and 10.